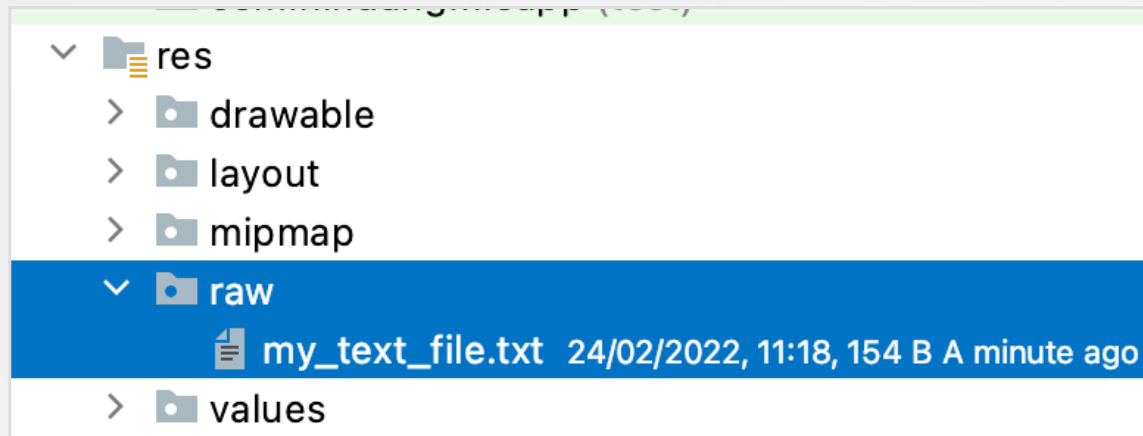
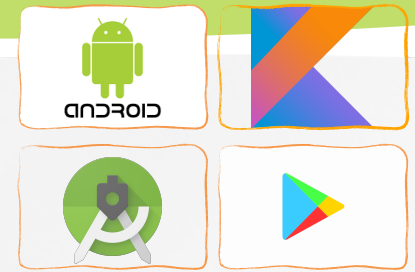


# Resource Files

## Example 1: read resource file

- » New > Folder > Raw resources folder
  - Add txt file ***my\_text\_file.txt*** into that folder





# Resource Files

## Example 1: read resource file

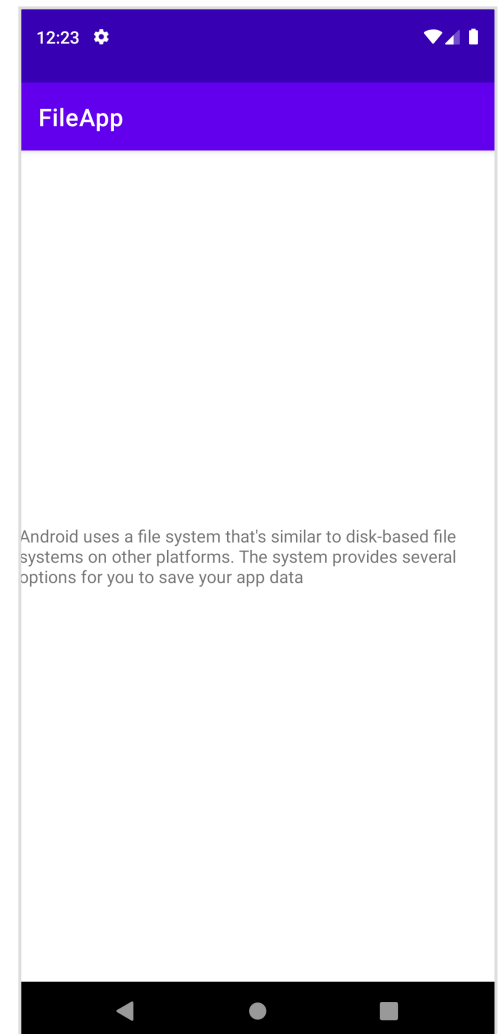
### » Read file's content in Main Activity

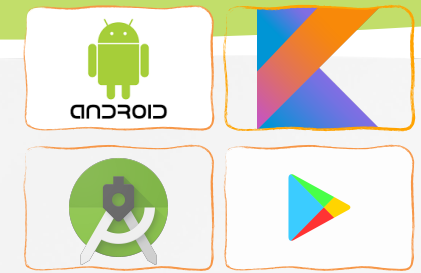
```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        var txtMsg = findViewById<View>(R.id.textView) as TextView
        try {
            txtMsg.setText(PlayWithRawFiles())
        } catch (e: IOException) {
            txtMsg.text = "Problems: " + e.message
        }
    }

    @Throws(IOException::class)
    fun PlayWithRawFiles() : String{
        val buf = StringBuffer()
        val iStream: InputStream = resources.openRawResource(R.raw.my_text_file)
        val reader = BufferedReader(InputStreamReader(iStream))

        if (iStream != null) {
            var line : String? = reader.readLine()
            while (line != null) {
                buf.append(line + "\n")
                line = reader.readLine()
            }
        }
        iStream.close()
        return buf.toString()
    }
}
```

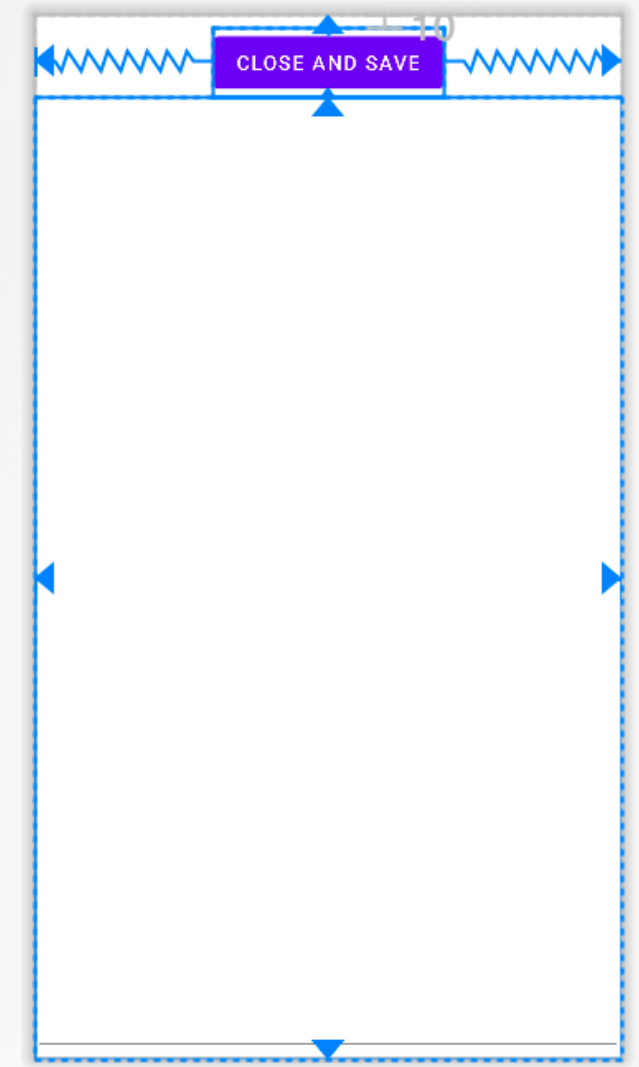


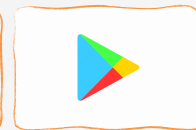


## Example 2

### Read/Write an Internal File

- » The application will collect data from the UI and save it to a persistent data file into the internal Android memory.
- Delete file
  - Read file's content onStart
  - Save file's content onPause



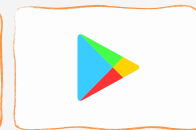


## Example 2

### Read/Write an Internal File

» Main Activity : editText, fileName and button event

```
class MainActivity : AppCompatActivity() {  
    val fileName = "test01.txt"  
    lateinit var editText: EditText  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        editText = findViewById(R.id.editText)  
  
        val btnFinish: Button = findViewById(R.id.saveBtn)  
        btnFinish.setOnClickListener {  
            finish()  
        }  
    }  
    ...  
}
```

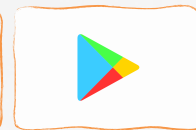


# Example 2

## Read/Write an Internal File

### » Main Activity: reading onStart()

```
override fun onStart() {  
    super.onStart()  
    try {  
        val inputStream: InputStream? = openFileInput(fileName)  
        if (inputStream != null) {  
            val inputStreamReader = InputStreamReader(inputStream)  
            val reader = BufferedReader(inputStreamReader)  
            val stringBuffer = StringBuffer()  
  
            var line : String? = reader.readLine()  
            while (line != null) {  
                stringBuffer.append(line + "\n")  
                line = reader.readLine()  
            }  
  
            inputStream.close()  
            editText.setText(stringBuffer.toString())  
        }  
    } catch (e: FileNotFoundException) {  
    } catch (t: Throwable) {  
        Toast.makeText(this, "Exception: $t", Toast.LENGTH_SHORT).show()  
    }  
}
```

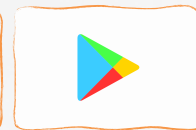


# Example 2

## Read/Write an Internal File

### » Main Activity: saving onPause()

```
// onPause
override fun onPause() {
    super.onPause()
    try {
        //File will be in "/data/data/$packageName/files/"
        val out = OutputStreamWriter(openFileOutput(fileName, 0))
        out.write(editText.getText().toString())
        out.close()
    } catch (t: Throwable) {
        editText.setText(t.message)
    }
}
```

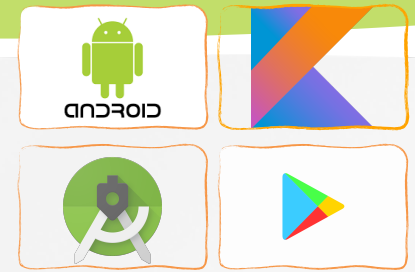


# Example 2

## Read/Write an Internal File

### » Main Activity: deleting file

```
private fun deleteFile() {  
    val packageName = applicationContext.packageName  
    val path = "/data/data/$packageName/files/$fileName"  
    val f1 = File(path)  
    Toast.makeText(applicationContext, "Exists " + f1.exists(),  
        Toast.LENGTH_SHORT).show()  
    val success: Boolean = f1.delete()  
    if (!success) {  
        Toast.makeText(applicationContext, "Deletion failed.",  
            Toast.LENGTH_SHORT).show()  
    } else {  
        Toast.makeText(applicationContext, "OK. File deleted.",  
            Toast.LENGTH_SHORT).show()  
    }  
}
```



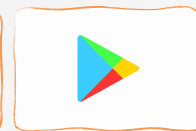
## Example 3

### Reading/Writing to the Device's **SD card**

» We need to request permission to read and write to the SD card by adding the following clauses to our `AndroidManifest.xml`

```
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>  
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```



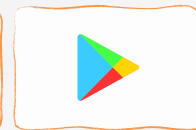


## Example 3

# Reading/Writing to the Device's **SD card**

» Main activity: fileName, editText, button's event, requesting permission...

```
class MainActivity : AppCompatActivity() {  
    val fileName = "test01.txt"  
    lateinit var editText: EditText  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        editText = findViewById(R.id.editText)  
  
        val btnFinish: Button = findViewById(R.id.saveBtn)  
        btnFinish.setOnClickListener {  
            if (ContextCompat.checkSelfPermission(  
                applicationContext, Manifest.permission.WRITE_EXTERNAL_STORAGE)  
                != PackageManager.PERMISSION_GRANTED  
            ) {  
                val permissions = arrayOf(Manifest.permission.READ_EXTERNAL_STORAGE,  
                    Manifest.permission.WRITE_EXTERNAL_STORAGE)  
                ActivityCompat.requestPermissions(this, permissions, 11111);  
            } else {  
                finish()  
            }  
        }  
    }  
}
```

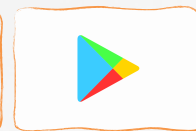


## Example 3

# Reading/Writing to the Device's **SD card**

## » Check permission result

```
override fun onRequestPermissionsResult(requestCode: Int,
    permissions: Array<out String>, grantResults: IntArray
) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)
    when (requestCode) {
        11111 -> {
            // If request is cancelled, the result arrays are empty.
            if (grantResults.size > 0
                && grantResults[0] == PackageManager.PERMISSION_GRANTED
            ) {
                finish() // finish and save file's content
            } else {
                Toast.makeText(this, "permission denied", Toast.LENGTH_SHORT).show()
            }
            return
        }
    }
}
```



# Example 3

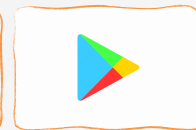
## Reading/Writing to the Device's **SD card**

### » Read file's content

```
fun readFile(){
    try {
        val path = "mnt/sdcard/$fileName"
        val file = File(path)
        val inputStream: InputStream? = FileInputStream(file)
        if (inputStream != null) {
            val inputStreamReader = InputStreamReader(inputStream)
            val reader = BufferedReader(inputStreamReader)
            val stringBuffer = StringBuffer()

            var line : String? = reader.readLine()
            while (line != null) {
                stringBuffer.append(line + "\n")
                line = reader.readLine()
            }

            inputStream.close()
            reader.close()
            editText.setText(stringBuffer.toString())
        }
    } catch (e: FileNotFoundException) {
    } catch (t: Throwable) {
        Toast.makeText(this, "Exception: $t", Toast.LENGTH_SHORT).show()
    }
}
```



## Example 3

# Reading/Writing to the Device's **SD card**

» Create file and write file's content

```
fun createandWriteFile(){
    try {
        val path = "mnt/sdcard/$fileName"
        val file = File(path)
        file.createNewFile()
        val outputStream = FileOutputStream(file)
        val writer = OutputStreamWriter(outputStream)

        writer.write(editText.getText().toString())

        writer.close()
        outputStream.close()
    } catch (t: Throwable) {
        editText.setText(t.message)
    }
}
```